

Présentation DevOps - TaskManager Microservices

1. Objectifs

Industrialiser le développement, automatiser les builds et déploiements, assurer la reproductibilité et la supervision.

2. Architecture globale

Le projet est composé d'un frontend Angular, d'une API Gateway, de microservices Auth, Task et AI, conteneurisés avec Docker puis déployés sur Kubernetes.

GitHub

Rôle :

GitHub est le dépôt de code source de l'application. C'est le point d'entrée de toute la chaîne DevOps.

Le code des différents modules y est stocké :

- Frontend Angular
- API Gateway
- Auth Service
- Task Service
- AI Review Service
- Dockerfile
- docker-compose.yml
- Fichiers Kubernetes
- Workflows GitHub Actions

Ce qui s'y passe

Lorsqu'un développeur :

- développe une nouvelle fonctionnalité,
- corrige un bug,
- modifie un fichier de configuration,

il effectue un :

```
git add  
git commit  
git push
```

Le **push** vers GitHub déclenche automatiquement toute la chaîne CI/CD.

GitHub Actions (CI/CD)

Après un **git push**, GitHub exécute automatiquement les workflows présents dans :

.github/workflows/

Par exemple :

ci.yml
cd.yml
ai-review.yml

(cités dans votre document)

Rôle

GitHub Actions automatise :

- la compilation Maven
- les tests unitaires
- la génération des artefacts
- la construction des images Docker
- leur publication
- le déploiement Kubernetes

Docker Compose

Une fois l'application compilée, les différents microservices sont exécutés localement grâce à **Docker Compose**.

Le document précise :

docker-compose.yml orchestre les conteneurs en local.

Dans votre projet, Docker Compose démarre simultanément :

- PostgreSQL
- Auth Service
- Task Service
- AI Review
- API Gateway
- éventuellement Angular

Son rôle

Docker Compose :

- construit les images Docker
- lance les conteneurs
- configure le réseau
- crée les volumes
- définit les variables d'environnement
- gère les dépendances entre services

Kubernetes

Une fois les images Docker créées et publiées, elles sont déployées sur Kubernetes.

Le document indique que Kubernetes utilise notamment :

- Namespace
- Deployment
- Service
- ConfigMap
- Secret
- Ingress

pour assurer le déploiement et l'exposition des services.

Son rôle

Kubernetes est l'orchestrateur des conteneurs.

Il permet de :

- déployer les applications,
- surveiller les conteneurs,
- redémarrer automatiquement un service en cas de panne,
- répartir la charge,
- mettre à jour les applications sans interruption.

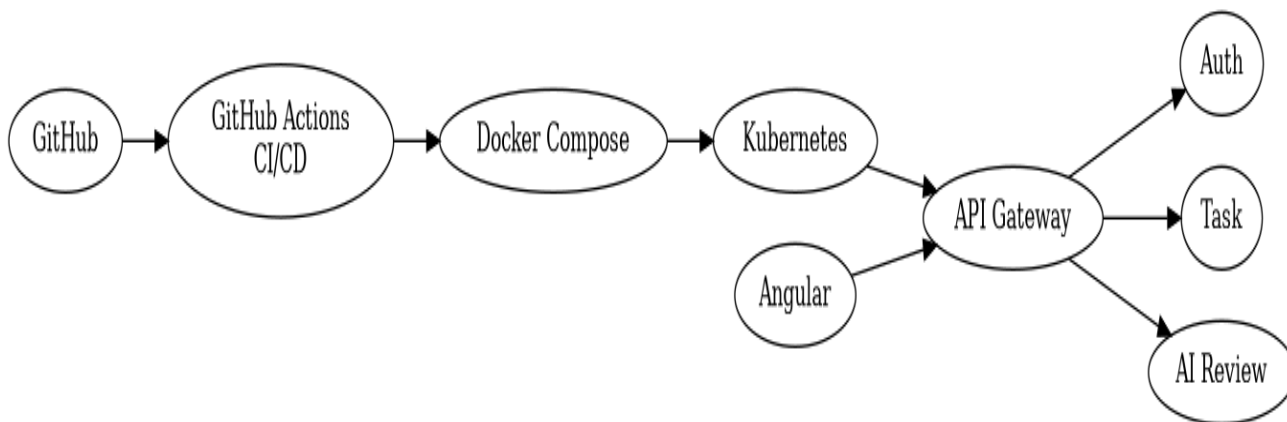


Figure 1 - Architecture DevOps simplifiée.

3. Docker

Dockerfile de chaque service, docker-compose.yml orchestre les conteneurs en local.

4. Kubernetes

Namespace, Deployment, Service, ConfigMap, Secret et Ingress assurent le déploiement et l'exposition.

5. GitHub Actions

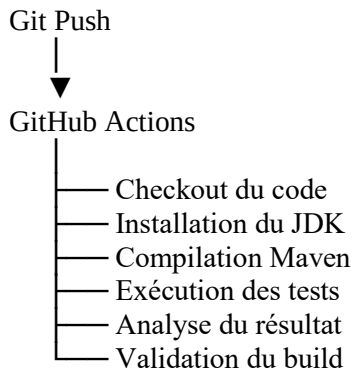
Workflows CI, CD et AI Review : compilation, tests, construction des images et déploiement.

Workflow CI (Continuous Integration)

Objectif

Le workflow **CI** vérifie automatiquement que le code ajouté par les développeurs est correct avant son intégration dans la branche principale.

À chaque **commit** ou **Pull Request**, GitHub Actions peut exécuter les étapes suivantes :

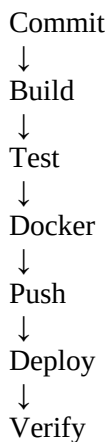


Workflow CD (Continuous Delivery / Deployment)

Objectif

Une fois la compilation et les tests validés par le workflow CI, le workflow **CD** automatise la création des images Docker et leur déploiement.

Le document décrit cette chaîne CI/CD comme :



Déroulement

CI validée

6. Scripts Python

devops_files.py prépare les fichiers et ressources. devops_ci.py automatise les étapes de build et d'intégration continue.

7. Configuration

.env, application.yml et fichiers Kubernetes centralisent le paramétrage.



Figure 2 - Chaîne CI/CD.

8. Déroulement d'un déploiement

- Commit Git
- Déclenchement GitHub Actions
- Compilation Maven
- Tests
- Construction Docker
- Publication des images
- Déploiement Kubernetes
- Validation

9. Points clés à présenter

- docker-compose.yml
- Dockerfile
- k8s/*.yml
- ci.yml / cd.yml / ai-review.yml
- devops_files.py
- devops_ci.py
- .env
- application-*.yml

• docker-compose.yml

Le fichier **docker-compose.yml** permet de lancer simultanément tous les conteneurs de l'application (API Gateway, microservices, bases de données, etc.) en environnement local. Il définit les images Docker, les ports, les volumes, les réseaux et les variables d'environnement de chaque service.

• Dockerfile

Le **Dockerfile** décrit les étapes nécessaires pour construire l'image Docker d'un service. Il indique notamment l'image de base, copie le code source, installe les dépendances et définit la commande de démarrage de l'application.

• k8s/*.yml

Les fichiers **k8s/*.yml** contiennent les ressources Kubernetes nécessaires au déploiement de l'application : **Deployment**, **Service**, **ConfigMap**, **Secret**, **Ingress**, etc. Ils permettent de déployer, configurer et exposer les microservices dans un cluster Kubernetes.

- **ci.yml / cd.yml / ai-review.yml**

Ces fichiers définissent les **workflows GitHub Actions**. **ci.yml** compile et teste l'application, **cd.yml** construit les images Docker puis déploie les microservices sur Kubernetes, tandis que **ai-review.yml** réalise une revue automatique du code à l'aide de l'intelligence artificielle.

- **devops_files.py**

Le script **devops_files.py** prépare automatiquement les fichiers nécessaires au déploiement DevOps. Il génère, copie ou organise les ressources et les fichiers de configuration utilisés par Docker, Kubernetes et GitHub Actions.

- **devops_ci.py**

Le script **devops_ci.py** automatise certaines étapes de l'intégration continue, comme la préparation des builds, les vérifications ou l'exécution de tâches techniques avant le lancement du pipeline CI/CD. Il simplifie et standardise le processus de déploiement.

- **.env**

Le fichier **.env** centralise les variables d'environnement de l'application, telles que les ports, les mots de passe, les URL des services ou les paramètres de base de données. Il permet de modifier la configuration sans changer le code source.

- **application-*.yml**

Les fichiers **application-*.yml** contiennent la configuration des applications Spring Boot. Ils définissent les paramètres propres à chaque environnement (développement, test, production), comme la connexion aux bases de données, les ports, la sécurité, les logs ou les profils Spring.

Ces fichiers constituent les principaux éléments techniques de votre chaîne DevOps :

- **Docker** assure la conteneurisation des applications.
- **Kubernetes** orchestre les conteneurs en production.
- **GitHub Actions** automatise les phases de build, de test et de déploiement.
- Les **scripts Python** facilitent les tâches répétitives.
- Les fichiers **.env** et **application-*.yml** centralisent toute la configuration de l'application.

10. Conclusion

Cette architecture met en œuvre une chaîne DevOps complète : automatisation, conteneurisation, orchestration Kubernetes et intégration continue.